

Dana ACSL

2024-2025

Computer science

2/21/25 Homework: [Problem 30](#)

My Google Colab

<https://colab.research.google.com/drive/1VprX4IOqaA04c1kkR8divL9AZj1Mywy?usp=sharing>

Print function

Syntax:

```
1 print("Hello, World")
```

- "Hello, World" can be replaced with anything
- You can also print numbers
 - Numbers don't have quotation marks

Comments

```
1  # This is a comment
```

- Comments are ignored by the program
- Start with a #
- Used to explain your code

Shortcut: ctrl + /

Data Types

Four basic data types:

- String - list of characters with quotation marks around it
 - Ex: "Hello", "10"
- Int - number with no decimal
 - Ex: 10, 5, 100
- Float - number with a decimal
 - Ex: 2.5, 1.1, 10.0
- Boolean - true or false

Variables

```
1 myString = "Hello, World"
```

Requirements for variable names:

- Start with letter or underscore (_)
- Can only contain letters, numbers, and characters
- Case sensitive - mystring and myString are different
- Can't be a python keyword

Printing Variables

```
1 myString = "Hello, World!"  
2 print(myString)
```

When printing variables, never have quotation marks around the name of your variable

What if you wrote this?

```
2 print("myString")
```

String Concatenation

- Use + to add two strings together

```
1 string1 = "Hello, "  
2 string2 = "World!"  
3 print(string1 + string2)
```

What is the type of
(string1 + string2)?

```
1 weather = "sunny"  
2 print("The weather today is " + weather)
```


String Concatenation

- Use + to add two strings together

```
1  string1 = 2
2  string2 = 4
3
4  print(string1+string2)
```

What will be printed when this program is run? What is the type of the two variables?

+ for two integers will add them together like in math

How can you change the variables for the print statement to print "24"?

What is wrong with this code

```
1  var1 = "number "  
2  
3  var2 = 2  
4  
5  var3 = var1+var2
```

+ with two variables will not work if one is a string and one is an int
However, you can use + to add a float and an int

Problem 1

Create a variable with a string of your name

Use a print statement to print "Hello my name is " + your name

Casting

- Cast a variable to another type
- `str()` - take an input and turn it into a string
- `int()` - take an input and turn it into an integer
- `float()` - take an input and turn it into a float (number with decimal)

If you input a decimal into `int()`, it will return that number rounded down

ex: `int(2.9)` will return 2

`int(2.3)` will return 2

Examples

```
x = str(2)  
print(x)
```

```
x = int(10.7)  
print(x)
```

```
x = int(1000)  
print(x)
```

```
x = float(1)  
print(x)
```

```
x = int(2.0)  
print(x)
```

```
x = float(1.1)  
print(x)
```

Problem 2

Recall the example from slide 8

How can you change var3 so that it will hold a string "number 2" and remove the error?

Hint: concatenation with a + requires two strings

```
1  var1 = "number "  
2  
3  var2 = 2  
4  
5  var3 = var1+var2
```

Strings

Strings in python are similar to lists (or arrays)

- You can use square brackets to get elements of the string
- Index refers to the position where an element in a string is located.
 - In python, indexing starts at 0

```
1 myString = "Hello, World!"  
2 print(myString[0])
```

This will print the first character in myString

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12

Length of a String

Use `len()` to find the length (how many characters) of a string

- Special characters, commas, exclamation marks etc. will be counted
- Spaces are also counted

```
1 myString = "Hello, World!"  
2 print(len(myString))
```

What is the length of the string?

What is the index of the last element (exclamation mark) in the string?

Index of last element in string = length of string - 1

Input

- Use input() to return information from the user
- If you want to use the information later in the program, assign it to a variable
- If you have a question to ask the user, put the question inside the parenthesis

```
input("What's your favorite food? ")
```

This asks the user for their favorite food

```
food = input("What's your favorite food? ")
```

This asks the user for their favorite food and stores it in a string called food

Whatever information the user inputs will be stored in a string

Problem 3

Write a program that asks the user for their name, then prints "Hello, " + name on a separate line

Problem 4

Write a program that asks the user for their name. Then print out the first initial of their name.

Problem 5

Write a program that asks the user for two integers, then outputs the sum and difference of the two integers.

If statements

If statements allow you to do more than one thing every time you run the program. Here is an example of an if statement.

```
a = 5
if a == 5:
    print("hi")
```

If the boolean expression is true, the code in the if statement will run. In this case the boolean expression is

```
a == 5
```

The code inside the if statement is

```
print("hi")
```

If statements (continued)

We can add an else after an if statement. The code in the else will run if the boolean expression is false.

```
name = "George"
if name == "Lennie":
    print("Small")
else:
    print("Milton")
```

In this case the boolean expression is

```
name == "Lennie"
```

The code inside the if statement is

```
print("Small")
```

The code inside the else statement is

```
print("Milton")
```

Problem 6

Write a program that asks the user for a password. If the password is “password” then print out “Welcome”. Otherwise, print out “Incorrect.”.

Elif statements

If the if statement is false, the program will check the elif statement

```
a = 5
if a == 10:
    print("a is 10")
elif a == 5:
    print("a is 5")
```

Similar to the if statement, elif statement will have a condition that needs to be met

Logical Conditions

<code>a == b</code>	Checks if a is equal to b
<code>a != b</code>	Checks if a is not equal to b
<code>a >= b</code>	Checks if a is greater than or equal to b
<code>a <= b</code>	Checks if a is less than or equal to b
<code>a > b</code>	Checks if a is greater than b
<code>a < b</code>	Checks if a is less than b

Double equals sign (==) checks for equality, single equals sign sets a variable equal to something

`a = 5`

Sets a equal to 5

`a == 5`

Checks if a is equal to 5
If it is, it will return True

Text adventure games!

You can combine input and if statements to create a simple text adventure game

General starting code:

```
decision = input("Your question goes here ")

if decision == "some answer":
    print("something happens")
elif decision == "some other answer":
    print("something else happens")
else:
    print("an else decision catches any invalid input")
```

Logical operators

and - returns True if both statements are true

or - returns True if at least one statement is true

not - returns False if result is true; returns True if result is false

```
a = 5
b = 10
if a == 5 and b == 100:
    print("Hello!")
if a == 5 or b == 100:
    print("Hello!")
```

First if statement will print "Hello!" if both conditions are true

Second if statement will print "Hello!" if at least one condition is true

```
a = 5
b = 10
print(not(a > 10))
```

a > 10 is false
not false is true
Will print True

.lower() and .upper()

.lower() and .upper() take a string and change it into lowercase or uppercase, respectively

```
a = input("Enter some text: ")  
print(a.lower())  
print(a.upper())
```

If you have an if statement and the condition deals with a user inputted string, it may be helpful to convert everything to lowercase or uppercase

```
print("LEFT".lower())  
print("Left".lower())  
print("LeFt".lower())  
print("left".lower())
```

These all print "left"

Escape characters

Use an escape character to type characters that are normally invalid in a string

```
print("Line1\nLine2")  
print('There\'s an apostrophe in this sentence')
```

First line: adds a new line

Second line: adds an apostrophe

- The \ is only necessary because the string is using single quotes
- The following would not need an escape character because it is using double quotes
- "There's an apostrophe in this sentence"

While loops

Loops let you run the same code over and over.
There are two kinds of loops: while loops and for loops.
Here is how you make a while loop:

```
1   i = 5
2   while i == 5:
3       print("Hi")
```

The code will run over and over until the boolean expression is false.
In this case the boolean expression is

```
i == 5
```

The code inside the while loop is

```
print("hi")
```

For loops

Loops let you run the same code over and over.
There are two kinds of loops: while loops and for loops.
Here is how you make a for loop:

```
for i in range(10):  
    print("hi")
```

This code creates a new variable `i`, and run the code underneath. Each time it runs the code, it will increase `i` by 1. This loop will end when `i` is equal to 10.

The code inside the while loop is

```
print("hi")
```

For loops (continued)

You can start with *i* being a number other than 0. In this example, *i* starts as 2. You can also change what *i* increments by each time. In this case, *i* increments by 3.

```
for i in range(2, 10, 3):  
    print(i)
```


Loops tips:

Use a for loop when you know exactly how many times the loop will run.
Otherwise, use a while loop!

Loops tips:

You can use the “break” statement to end a loop early!

```
for i in range(2,10,3):  
    print(i)  
    break
```

```
for i in range(10):  
    if (i == 3):  
        continue  
    print(i)
```

You can use the “continue” statement to go onto the next iteration (run) of the loop.

Problem 7

Print out all the numbers from 1 to 100. Hint: Use a for loop!

Problem 8

Ask the user for a password. Don't stop asking them until they enter "password".
Hint: Use a while loop!

Modulo (%)

You can use the % symbol for modulo.

What is modulo? Modulo is dividing and taking the remainder.
Before decimals, you learned remainders.

$a \% b$

Will give you the remainder when a is divided by b .

```
print(10%3)
```

What will this print out?

Modulo (%) (continued)

What can you use modulo for?

To find if something is divisible by another thing.

If $a \% b = 0$, then a is divisible by b .

```
for i in range(10):  
    if (i%3 == 0):  
        print(i)
```

This code prints out all the numbers divisible by 3 from 0 to 9, inclusive.
(inclusive means including 0 and 9)

Problem 9

Ask the user for a number. Print out all the numbers from 0 to that number (inclusive) that are divisible by 3 or 5, but not both.

Problem 10

Use a for loop to print out all the characters in the string "Hello, World!".
Hint: to print a specific character of a string: `stringName[index]`

Functions

- Block of user defined code that runs when called
- Can return information
- Created with the def keyword

```
def myFunction():  
    print("This is a function")
```

- Write the function name with parentheses to call the function

```
myFunction()
```

Use functions if you have blocks of code that repeat multiple times in your program

Arguments

- Can pass information into functions as arguments
- Write arguments inside parentheses separated by commas

```
def myFunction(name):  
    print("Hello " + name)  
  
myFunction("Bob")
```

Return Values

- Use the return statement to return a value from a function

```
def myFunction(num):  
    return num+1
```

Problem 11

Create a function that takes in two arguments and prints the sum of the two numbers. Call the function with the values 2 and 5.

Recursive Functions

- Function that calls itself

```
def myFunction(x):  
    print(x)  
    if x == 0:  
        print("Base Case")  
        return  
    else:  
        return myFunction(x-1)
```

- First if statement = base case
 - If x is 0, print "Base case" and return
- Recursive functions need base cases or else they will run forever

Factorial with Recursion

Factorial = product of every number less than or equal to number inputted

- $4! = 4*3*2*1$
- $5! = 5*4*3*2*1 = 5*4!$

```
def factorial(x):  
    if x == 1:  
        return 1  
    elif x > 1:  
        return x * factorial(x-1)  
    else:  
        return "Factorial doesn't exist for negative numbers"
```

Problem 12

Fibonacci sequence is when each number is the sum of the two numbers before it: 0, 1, 1, 2, 3, 5, 8, 13...

Create a recursive function that returns the value of the fibonacci sequence at the number inputted.

- If 0 is inputted, return 0
- If 1 is inputted, return 1
- If 2 is inputted, return 1
- If 3 is inputted, return 2
- If 4 is inputted, return 3

Problem 13

def function(*x): ...

- * allows the function to take in more arguments than specified
- x can be iterated through similar to a list or string (for i in x: ...)

Create a function that finds the sum of all the numbers inputted as arguments where the number of numbers is not specified

Problem 14

Write a recursive function based on the image below. Call the function with the value 19.

- Find $f(19)$ given below:

$$f(x) = \begin{cases} x + f(x - 3) & \text{if } x \geq 15 \\ 2 * f(x + 2) & \text{if } 12 < x < 15 \\ x + 2 & \text{if } x \leq 12 \end{cases}$$

Problem 15

Write a recursive function that takes in a number as input and then prints a series of numbers with each number on a separate line. The series starts with the inputted number, then the inputted number with the last digit removed, the inputted number with the last two digits removed and so on.

For example, if the user inputs 1032904, the output should be:

```
1032904
103290
10329
1032
103
10
1
```

To get the result of division without a remainder, use //

Lists

- One variable that holds multiple values
- Create with square brackets

```
fruits = ["apple", "banana", "grapes"]
```

- Can have different data types and duplicate values

```
list = ["apple", 10, True, "apple"]
```

- Access items by index
 - First item is index 0, second item is index 1...
- Find length of list with len() function

List Methods

`append()` - adds an item to the end of a list

`insert()` - adds an item at the specified position

- Takes in two parameters: position to insert and the item to insert

`pop()` - removes the item at the specified position

- If there is no position specified, it deletes the last element

`remove()` - removes the first item with the specified value

`sort()` - sorts the list in ascending or alphabetical order by default

- `reverse=True` sorts the list descending
- Can take in a key, user defined function to sort

Problem 16

Create an empty list. Ask the user for a string then add it to the end of your list. Ask the user again for a string and append it to the end of the list. Continue adding the user's input to the list until the list has 5 elements in it. Lastly, print the list.

Looping through a list

- Use a for loop to loop through a list

```
list = ["apple", "strawberry", "banana", "grapes"]  
  
for i in range(len(list)):  
    print(list[i])
```

- Can loop through each element in the list directly instead of looping through the index

```
for i in list:  
    print(i)
```

Problem 17

```
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Write a program that loops through this list and checks if each element is even. If an element is even, append it to a new list.

List Comprehension

- Faster way to create new list

Syntax: `newlist = [expression for item in iterable if condition == True]`

```
newlist = [i for i in list if i%2 == 0]
```


Problem 18

Create a list of strings. Use list comprehension to create a new list from the list of strings. If the string has an e in it, add it to the new list. Then print your list.

Dictionaries

```
dictionary = {  
    "name": "Bob",  
    "age": 50,  
    "height": "6ft"  
}  
  
print(dictionary["name"])
```

- key: value
- key is an identifier for a value
- Can't have two items with same key
- Access values with the key in square brackets
 - Can also use .get()

```
print(dictionary.get("name"))
```

Dictionaries

```
print(dictionary.keys())  
  
print(dictionary.values())  
  
print(dictionary.items())
```

- .keys() returns list of all keys
- .values() returns list of all values
- .items() returns list of all keys and values in a tuple

Problem 19

Write a program that uses a for loop to iterate through every key in a dictionary and then print it out.

Adding to a dictionary

```
dictionary["phone"] = 1234567890
```

- Adds a phone number to the dictionary
 - key = "phone", value = 1234567890
 - Can also change pre existing values
- To add a new value, create new index and assign value

```
dictionary.update({"name": "Bill", "phone": 1234567890})
```

- .update() changes multiple values
 - Takes in dictionary containing everything to be changed or added

Deleting from a dictionary

- `del`
 - Deletes item with key

```
del dictionary["height"]
```

- `.pop()`
 - Takes in key of value to be deleted
 - Deletes the value but also returns it
 - Can set it equal to a variable for use later in code

```
dictionary.pop("height")
```

```
height = dictionary.pop("height")
```

- `.clear()` clears the entire dictionary
 - Dictionary still exists, but has nothing inside

```
dictionary.clear()
```

Problem 20

```
dictionary = {  
    "name": "Bob",  
    "age": 50,  
    "height": "6ft",  
    "phone": 1234567890  
}
```

Remove the keys "age" and "height" from this dictionary. Then print the dictionary.

Problem 21

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}  
dict2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}
```

Write a program that merges these two dictionaries together into one dictionary. Print the dictionary

Problem 22

```
sampleDict = {  
    "class": {  
        "student": {  
            "name": "Mike",  
            "marks": {  
                "physics": 70,  
                "history": 80  
            }  
        }  
    }  
}
```

Write a program that prints out the value in the key "history".

Tuples

```
fruits = ("apple", "banana", "pear")
```

- Ordered
 - Can use index to access an element
- Unchangeable

```
fruitsTuple[0] = "strawberry"  
print(fruitsTuple)
```

Results in an error

- can have different data types
- can have duplicates
- Generally faster than lists

Use tuples when you don't want the data inside to change

Tuples

`print(len(tupleName))` - prints the length of the tuple

`print("value" in tupleName)` - print True if "value" is in tupleName, prints False otherwise

Tuple methods

`print(tupleName.index("value"))` - prints the index of value

`print(tupleName.count("value"))` - prints out how many "value"s in the tuple

Unpacking tuples

```
fruitsTuple = ("apple", "banana", "pear")  
  
(fruit1, fruit2, fruit3) = fruitsTuple
```

- “Unpacking” means extracting values
- Number of variables matches number of values in the tuple
- fruit1, fruit2, and fruit3 become variables that you can use later in the code

```
fruitsTuple = ("apple", "banana", "pear", "strawberry", "blueberry")  
  
(fruit1, fruit2, *fruit3) = fruitsTuple
```

- “Unpacking” means extracting values
- Number of variables matches number of values in the tuple
- fruit1, fruit2, and fruit3 become variables that you can use later in the code

Problem 23

Create a tuple with three strings in it. Write a program that changes the first value in the tuple to a new string.

Hint: What other collection type similar to tuples are changeable?

Problem 24

`t = (1, 2, 2, 3, 4, 7, 2, 9, 8, 5, 2)`

Use tuple methods to print the number of times 2 appears in `t` and print the index of 9.

Problem 25

`t = (5, 10, 15, 20, 25, 30, 35, 40)`

Write code that gets rid of all the odd numbers in this tuple.

Problem 26

```
ages = [("Bob", 25), ("Todd", 30), ("Charlie", 35)]
```

Use a loop to unpack and print a sentence for each value in the ages list.
For example, the first thing printed should be "Bob is 25 years old"

Problem 27

Define a function `areaPerimeter` that takes in two integers, `length` and `width`, and returns a tuple with the area and the perimeter of the rectangle with the given `length` and `width`.

Sets

```
set1 = {"apple", 2, True, "banana"}
```

- Unordered
- Unchangeable
 - However, can add/delete values
- No duplicates allowed
 - 1 and True are considered duplicates, as well as 0 and False
- Can be iterated through using loop
- Cannot use index to access items

Basic set methods

`set1.add("grape")` - adds "grape" to set1

`set1.remove("apple")` - removes "apple" from set1

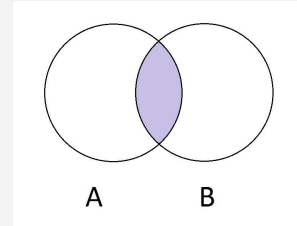
`set1.pop()` - removes an item from set1

`set1.clear()` - clears set1

More set methods

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

set3 = set1.intersection(set2)
print(set3)
```



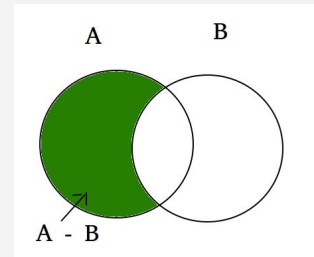
.intersection creates a set with the items that are in both set1 and set2

- Can use & as shortcut (set3 = set1 & set2)

```
set3 = set1.difference(set2)
```

.difference creates a set with the values in set1 that are not in set2

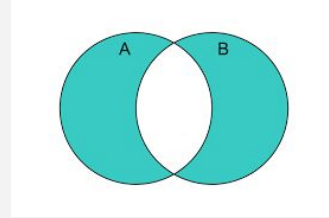
- If all values overlap, returns empty set
- Shortcut: set3 = set1 - set2



More set methods

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

set3 = set1.symmetric_difference(set2)
print(set3)
```



`.symmetric_difference` creates a new set with all the values that don't overlap in `set1` and `set2`

Problem 24

numbers = [1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]

Write code that deletes the duplicate numbers from this list.

Problem 25

```
hasDog = {"Bob", "Olivia", "Tim", "James"}  
students = {"James", "Mary", "John", "Olivia", "Tim"}  
playsSports = {"Bob", "Mary", "Susan", "Tim"}
```

Use set methods to:

- a) Print the names of the people who are students but don't play sports
- b) Print the names of the people who either have a dog or play sports
- c) Print the names of the people who have dogs and are students
- d) Print the name of the person who has a dog, is a student, and plays sports

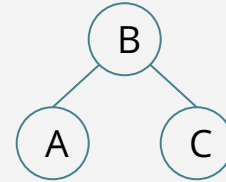
Creating a binary search tree using lists

This is one node:

[left, value, right]

None means no value

Eg. [[None, 'A', None], 'B', [None, 'C', None]]



Three cases when adding a node to a binary search tree

- If there is no root, binary search tree is empty
- If value to add is greater than root, move right
- If value to add is less than or equal to root, move left

Creating a binary search tree using lists

- If there is no root, put the value to add in the middle of the list
 - [None, None, None] becomes [None, value, None]
- If the value is greater than the root, move to the right (eg. value = "D")
 - "D" > "B"
 - [[None, "A", None], "B", [None, "C", None]] instead of looking at "B",
 - [[None, "A", None], "B", [None, "C", None]] move right to look at "C"
- If the value is less than or equal to the root, move to the left (eg. value = "A")
 - "A" < "B"
 - [[None, "A", None], "B", [None, "C", None]] instead of looking at "B"
 - [[None, "A", None], "B", [None, "C", None]] move left to look at "A"
- What if I want to look to the left (or right) but there is no node there?
 - [None, "A", [None, "B", None]], value to add = "A"
 - "A" <= "A"

Creating a binary search tree using lists

- If you want to look to the left or the right but there is no new node there, create a new empty node and add the value
 - For these examples assume that the value we are adding is "D"
 - [None, "A", None] becomes [None, "A", [None, "D", None]]
 - [None, "A", [None, "B", None]] becomes [None, "A", [None, "B", [None, "D", None]]
 - Notice how in both cases, we add a [None, None, None] and then put the value ("D") inside the new node.
- When the binary search tree is empty, [None, None, None]
- After we add a new node, the new node will be [None, None, None]
 - In both cases when we add a new value, the value will go into the 1st index of the list (that we use to represent a node)
- Use this pattern to write a recursive function where the base case is if the node is empty

Creating a binary search tree using lists

Let's look at an example:

string = "DABC"

- originally root = [None, None, None]
- Since the root is empty, we put the "D" into the 1st index of root
 - root = [None, "D", None]
- Next, compare "A" with "D"
 - Since "A" < "D", we move to the left
root = [None, "D", None]
 - There is no node to the left of "D" so we add a new empty node
root = [[None, None, None], "D", None]
 - Since this node is empty, we can place "A" into it
root = [[None, "A", None], "D", None]
- Compare "B" with "D"
 - Since "B" < "D", move to the left and compare "B" with "A"
root = [[None, "A", None], "D", None]
 - Since "B" > "A", move to the right
root = [[None, "A", None], "D", None]
 - There is no node to the right of "A" so we create an empty node
root = [[None, "A", [None, None, None]], "D", None]
 - The node is empty so we can put "B" inside it
root = [[None, "A", [None, "B", None]], "D", None]

Creating a binary search tree using lists

Continuing from last slide:

string = "DABC"

root = [[None, "A", [None, "B", None], "D", None]

- "C" < "D" so move left to compare "C" with "A"

root = [[None, "A", [None, "B", None], "D", None]

- "C" > "A" so move right to compare "C" with "B"
- root = [[None, "A", [None, "B", None], "D", None]
- "C" > "B" so move right

root = [[None, "A", [None, "B", None], "D", None]

- There is no node to the right of "B" so create a new empty node

root = [[None, "A", [None, "B", [None, None, None]], "D", None]

- Since the node is empty, place "C" in the empty node

root = [[None, "A", [None, "B", [None, "C", None]], "D", None]

Pattern:

- Compare the value to add with the value in the node
- Move right/left
- Eventually there will be no more nodes
- Add a new empty node
- Place the value to add into the node

If you are confused read the speaker notes

Problem 26

Look at the code for creating a binary search tree in the speaker notes. (or try writing it by yourself!)

Change `s` to a few different strings and try to follow what the function is doing when `addNode()` is called. (think what does this program do)

Write a new function called `printBST()` that takes in one parameter, the name of the binary search tree (in this case, `root`). `printBST` should print all the values in the binary search tree in alphabetical order.

For example:

`s = "OLYMPIAD"`

After turning it into a binary search tree, `root = [[[[None, 'A', [None, 'D', None]], 'I', None], 'L', [None, 'M', None]], 'O', [[None, 'P', None], 'Y', None]]`

`printBST()` should print A D I L M O P Y each on its own line

Problem 27

Write a function, `printBST2`, that takes in one parameter, the binary search tree and prints out all the values in it in descending order. For example, instead of printing A D I L M O P Y from problem 26, it will print Y P O M L I D A.

Problem 28

Write a function, `deleteValue`, that takes in two parameters: the value to delete, and the name of the binary search tree. If the node has no children, delete the node. If the node has one child, replace the node with its child and delete the node. If the node has two children, replace the node with its left child and stick the right child onto the bottom right of the left child. If the value to delete doesn't exist in the tree, return the tree with nothing changed.

Classes

- Blueprint for creating instances

```
class Class1:  
    greeting = "Hello World!"  
  
person = Class1()
```

- person is an instance of Class1
- person.greeting will print "Hello World!"
- Can create instance variables
- Attributes are variables that hold data
- Methods are functions that belong to the class

__init__() function

- Used to assign values
- Called automatically when class instance is created

```
class Car:  
    def __init__(self, topSpeed, color, price):  
        self.topSpeed = topSpeed  
        self.color = color  
        self.price = price
```

- self is similar to a placeholder for whatever is being created

__str__() function

- Defines what should be printed when object is printed as a string

```
def __str__(self):  
    return self.color + " car"
```

- Will print the color of the car

Problem 29

Create a class, Student that has an `__init__` function and a `__str__` function. In the `init` function, assign values for the student's name, age, and grade. In the `str` function, return a string that says (student's name) is (student's age) years old and in grade (student's grade).

Create two instances of the student class, `student1` and `student2` with whatever values for name, age, and grade you want. Then print `student1` and `student2`.

Class variables

- Class variable shared by all instances of the class
 - Defined outside `__init__()` method
 - Stores data that is the same for all instances
- Instance variable is defined inside `__init__()` method
 - Is specific to each instance
 - Stores data that is unique to each instance

Use `car1.__dict__` to see everything inside `car1`

Compare it to `Car.__dict__`

```
class Car:
    carCount = 0
    def __init__(self, topSpeed, color, price):
        self.topSpeed = topSpeed
        self.color = color
        self.price = price
        Car.carCount += 1
```

Class variables

- carCount can be accessed with instance names and class name
 - When accessing attribute of instances, first checks if the instance has the attribute
 - If it doesn't, it checks if the class contains the attribute
 - Instances (car1 and car2) don't actually have carCount variable
 - Accesses carCount variable from the Car class

```
print(car1.carCount)  
print(car2.carCount)  
print(Car.carCount)
```

All prints 2

```
car1.carCount = 3
```

What will this change when we print carCount using the three lines above?

Class variables

```
car1.carCount = 3
```

- When we assigned `car1.carCount` to 3, created `carCount` variable inside `car1`
- When we print `car1.carCount`, computer already found a `carCount` variable inside the instance
 - Doesn't go look at `Car.carCount` variable
- Didn't assign `car2.carCount` to anything
 - When we print `car2.carCount`, prints `carCount` variable from `Car` class

Inheritance

- Parent class is the class being inherited from
 - Any class can be parent class
- Child class is the class that is inheriting from the parent class
 - To create child class, pass in parent class as a parameter when creating the child class

```
class Vehicle:  
    pass  
  
class Car(Vehicle):  
    pass
```

- Car is the child class, which inherits properties from Vehicle, the parent class

Inheritance

- If no `__init__` function defined in child class, child class will use parent's `__init__` function
- When `__init__` function is added to child class, will override `__init__` function defined in parent class

Problem 30

Create two classes. In the first class, Shape, create an init function that defines the length and width of the shape. The second class, Rectangle, should be a child class of Shape.

Create a Shape and a Rectangle and initialize it with whatever values you want.

Super

- Allows you to refer methods defined in the parent class from the child class.

```
class Parent:
    def __init__(self, txt):
        self.message = txt

class Child(Parent):
    def __init__(self, txt):
        super().__init__(txt)
```

- Everything inside init function of Parent class goes into init function of child class

Problem 31

```
class Person:
    def __init__(self, name):
        self.name = name
# Write code here
```

Create an Employee class that is a child of the Person class. In the init method of the Employee class, take in a name and id. Initialize the name and id using super()

Iterators

- Iterable - something that you can loop over: lists, tuple, dictionary...
 - Contains iter method
 - Something can be looped over if it contains iter method
- Iterator - object that keeps track of where it is in the sequence and knows how to get the next value
 - Uses next method to get next value
 - When there are no more values, raises stop iteration exception

```
fruits = ["apple", "banana", "strawberry"]  
fruitIterator = iter(fruits)  
print(next(fruitIterator))  
print(next(fruitIterator))
```

Returns iterator from the list and prints the first two values

Iterators

- To create an iterable, define a class with an iter method
 - When iter is called, an iterator will be provided
- Iterators need iter and next method
 - In next method, StopIteration should be raised if there are no more values

Problem 32

Write a python class Reverse that takes a string and iterates through it in reverse order, meaning the elements should be returned starting with the last element and ending with the first. The class should accept a string when it is initialized.

Implement the `__iter__` method to initialize the iteration process and the `__next__` method to get the next method in the iteration. When all the elements have been returned, the next method should raise the `StopIteration` exception to signal the end of the iteration.

Generators

- Similar to functions but yield a value instead of returning value
- Also iterators but iter and next methods are created automatically

```
def numbers(start):  
    current = start  
    while True:  
        yield current  
        current += 1
```

Generates infinite sequence of numbers starting from the start value

- Yield pauses the function, returns a value, and then continues the function again
 - Can continue from the same point
- Since generators don't store every value, they are more memory efficient than storing a long list of values

Generators

```
def count(end):  
    current = 1  
    while current <= end:  
        yield current  
        current += 1  
  
a = count(10)  
for i in a:  
    print(i)
```

```
a = count(10)  
for i in range(10):  
    print(next(a))
```

- Can loop over a generator like with a list
- next() function retrieves next value from a generator

Both will print all the integers from 1 - 10 on a separate line

Generator expressions

- Generator expressions are a more concise way to create a generator object
- Similar to list comprehension
 - List comprehension gives you immediate access to everything in the list
 - Generator expression only gives one value at a time but is more memory efficient
- Syntax: (expression for item in iterable)

```
gen = (i for i in range(10))  
  
for i in gen:  
    print(i)
```

Problem 32

Write a generator function that generates factorial numbers indefinitely.

The factorial of a number is the product of all positive integers less than or equal to that number. For example, $5! = 5 * 4 * 3 * 2 * 1 = 120$

Problem 33

A triangular number is a number obtained by adding together all positive integers less than or equal to a positive integer. The first few triangular numbers are 1, 3, 6, 10, 15...

Write a generator function that generates triangular numbers indefinitely.

Decorators

- Design pattern that enhances original function without changing actual code
- Used to add common behavior to functions that can be reused
- Function that takes in a function as input and returns new function

```
def simple_decorator(func):  
    def wrapper():  
        print("Before function call")  
        func()  
        print("After function call")  
    return wrapper
```

Returns wrapper function
wrapper function will print two
lines of code before and after
calling the original function

Decorators

- Apply the decorator using @ symbol
 - @simple_decorator is the same as greet = simple_decorator(greet)

```
@simple_decorator
def greet():
    print("Hello!")
greet()
```

```
def decorator(function):
    def wrapper(*args, **kwargs):
        print("Before function")
        function(*args, **kwargs)
        print("After function")
    return wrapper
```

- To take in arguments, add *args, **kwargs as parameters
- *args collects any extra arguments into a tuple
- **kwargs collects any extra keyword arguments into a dictionary